

AD Module — Architecture

Author: Andrews Ferreira

Version: 2.0 (modularized)

Overview

The AD module performs Active Directory reconnaissance through a five-phase kill chain. The modular architecture separates concerns into four packages that form a clear data pipeline:

```
collectors → analyzers → attack_paths → reporting
```

The **phases** are thin orchestration layers that wire these packages together. The **orchestrator** (`ad_module.py`) manages tool/parser lifecycle, phase sequencing, and final report generation.

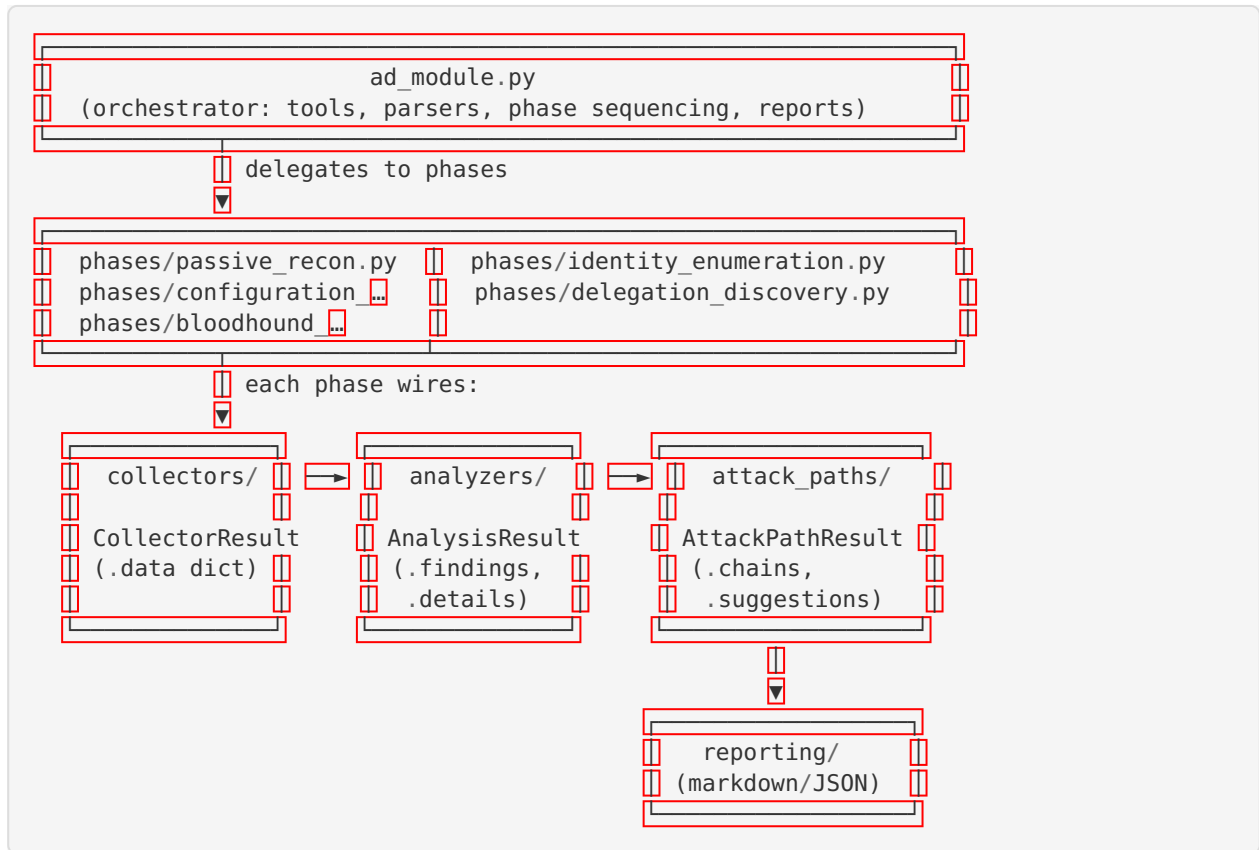
Package Map

```

modules/ad/
├── ad_module.py           # Orchestrator (< 500 lines)
├── base.py               # ADPhaseBase ABC (shared by phases)
├── ARCHITECTURE.md       # This file
├──
├── collectors/          # Data acquisition layer
│   ├── base.py          # CollectorBase ABC + CollectorResult
│   ├── ldap_collector.py # LDAP queries (users, groups, SPNs, GPOs, ...)
│   ├── smb_collector.py  # SMB null sessions, shares, enum4linux-ng
│   ├── kerberos_collector.py # Kerberos detection, AS-REP, RID cycling
│   ├── dns_collector.py  # AD service scan, SRV records
│   ├── delegation_collector.py # Delegation enumeration (unconstrained, constrained, RBCD)
│   └── bloodhound_collector.py # Bloodhound-python + netexec fallback
├── analyzers/           # Analysis & enrichment layer
│   ├── base.py          # AnalyzerBase ABC + AnalysisResult
│   ├── permission_analyzer.py # SMB signing, anon LDAP, null sessions, share ACLs
│   ├── relationship_analyzer.py # Group memberships, privileged users, DCs
│   ├── misconfiguration_analyzer.py # Password policy, delegation, Kerberos, MAQ
│   ├── privilege_analyzer.py # Bloodhound HVTs, DA, kerberoastable, AS-REP
│   └── trust_analyzer.py  # Trust relationships and risk assessment
├── attack_paths/        # Offensive chain construction
│   ├── base.py          # AttackPathBuilderBase + AttackChain / NextStepSuggestion
│   ├── kerberoast_paths.py # Kerberoasting chains
│   ├── asrep_paths.py    # AS-REP roasting chains
│   ├── delegation_paths.py # Unconstrained / constrained / RBCD abuse
│   ├── gpo_paths.py      # GPO & SYSVOL credential hunting
│   ├── acl_paths.py      # Bloodhound ACL paths, session theft, relay
│   └── privilege_escalation_paths.py # Password spray, trust exploitation
├── reporting/           # Output & report generation
│   ├── base.py          # ReporterBase ABC
│   ├── attack_surface_reporter.py # Executive summary / quick report
│   ├── high_value_targets_reporter.py # HVT report (DA, SPNs, AS-REP, delegations)
│   ├── attack_path_reporter.py # Attack paths grouped by risk
│   ├── remediation_reporter.py # Remediation recs by severity
│   ├── ad_summary_reporter.py # Legacy AD summary markdown
│   └── report_builders.py # Data builder functions for reporters
├── phases/              # Thin orchestration phases
│   ├── passive_recon.py  # Phase 1 – service discovery, anon access
│   ├── identity_enumeration.py # Phase 2 – users, groups, SPNs, AS-REP
│   ├── configuration_enumeration.py # Phase 3 – policies, trusts, GPOs
│   ├── delegation_discovery.py # Phase 4 – delegation enumeration
│   └── bloodhound_collection.py # Phase 5 – graph collection
├── tools/               # Tool wrappers (unchanged)
│   ├── nmap.py, ldapsearch.py, smbclient.py, impacket.py,
│   └── enum4linux_ng.py, bloodhound.py, netexec.py, advanced_impacket.py
├── parsers/             # Output parsers (unchanged)
│   ├── nmap_parser.py, ldap_parser.py, smb_parser.py, impacket_parser.py,
│   ├── enum4linux_ng_parser.py, bloodhound_parser.py, netexec_parser.py,
│   └── delegation_parser.py

```

Data Flow



Data Contracts

Boundary	Type	Key Fields
Collector → Phase	CollectorResult	success, data (dict), errors
Phase → Analyzer	raw dicts / lists	protocol-specific keys
Analyzer → Phase	AnalysisResult	findings (list), details (dict), risk_level
Phase → AttackPathBuilder	context dict	technique-specific keys
AttackPathBuilder → Phase	AttackPathResult	chains (AttackChain list), suggestions
Orchestrator → Reporter	builder dicts	see report_builders.py

Phase Responsibilities

#	Phase	Collectors Used	Analyzers Used	Attack Path Builders
1	Passive Recon	Dns, Ldap, Smb, Kerberos	Permission	—
2	Identity Enumeration	Ldap, Kerberos, Smb	Relationship, Misconfiguration	Kerberoast, As-rep
3	Configuration Enumeration	Ldap, Smb	Misconfiguration, Permission, Trust	Gpo, PrivilegeEscalation
4	Delegation Discovery	Delegation	Misconfiguration	Delegation
5	Bloodhound Collection	Bloodhound	Privilege	Kerberoast, As-rep, Acl

Extending the Module

Adding a New Collector

1. Create `collectors/my_collector.py`
2. Subclass `CollectorBase` and implement `collect(self, **kwargs) → CollectorResult`
3. Export from `collectors/__init__.py`
4. Use in the relevant phase(s)

```
# collectors/my_collector.py
from modules.ad.collectors.base import CollectorBase, CollectorResult

class MyCollector(CollectorBase):
    def __init__(self, tool, parser, logger):
        super().__init__(logger)
        self.tool = tool
        self.parser = parser

    def collect(self, target: str, **kwargs) -> CollectorResult:
        raw = self.tool.run(target)
        parsed = self.parser.parse(raw)
        return CollectorResult(success=True, data={"my_data": parsed})
```

Adding a New Analyzer

1. Create `analyzers/my_analyzer.py`
2. Subclass `AnalyzerBase` and implement `analyze(self, data, **kwargs) → AnalysisResult`
3. Export from `analyzers/__init__.py`

Adding a New Attack Path

1. Create `attack_paths/my_paths.py`
2. Subclass `AttackPathBuilderBase` and implement `build(self, context, **kwargs) → AttackPath-Result`
3. Build `AttackChain` objects with steps, risk level, prerequisites, and references
4. Export from `attack_paths/__init__.py`

Adding a New Reporter

1. Create `reporting/my_reporter.py`
2. Subclass `ReporterBase` and implement `generate(self, data) → str`
3. Add a `build_my_data()` function in `report_builders.py`
4. Wire in `ad_module.py._generate_reports()`

Design Decisions

- **Phases are thin:** They contain only orchestration logic — no data parsing, finding generation, or report building. All domain logic lives in the four packages.
- **Duck-typing for parser objects:** Analyzers use `getattr()` on collector results so they can accept raw parser dataclasses without coupling to specific parser implementations.
- **Tools and parsers are unchanged:** The `tools/` and `parsers/` packages are stable, well-tested layers. The modularization only restructured the code above them (phases, analysis, attack paths, reporting).
- **Report builders are free functions:** `report_builders.py` contains pure functions that assemble data dicts from the orchestrator's state. This keeps the orchestrator under 500 lines and makes the builders independently testable.
- **500-line limit:** Every file stays under 500 lines to keep cognitive load manageable and code reviewable.

Preserved Capabilities

All original offensive capabilities are preserved:

Capability	Original Location	New Location
Anonymous LDAP bind detection	passive_recon.py	collectors/ ldap_collector.py → analyzers/permission_analyzer.py
SMB null session testing	passive_recon.py	collectors/smb_collector.py → analyzers/permission_analyzer.py
Kerberos service detection	passive_recon.py	collectors/kerberos_collector.py
User / group / computer enum	identity_enumeration.py	collectors/ ldap_collector.py
SPN / Kerberoasting	identity_enumeration.py	collectors/ ldap_collector.py → attack_paths/kerberoast_paths.py
AS-REP roasting	identity_enumeration.py	collectors/kerberos_collector.py → attack_paths/asrep_paths.py
Password policy analysis	configuration_enumeration.py	collectors/ ldap_collector.py → analyzers/misconfiguration_analyzer.py
Trust enumeration	configuration_enumeration.py	collectors/ ldap_collector.py → analyzers/trust_analyzer.py
GPO / SYSVOL hunting	configuration_enumeration.py	collectors/ ldap_collector.py → attack_paths/gpo_paths.py
Unconstrained delegation	delegation_discovery.py	collectors/delegation_collector.py → attack_paths/delegation_paths.py
Constrained delegation (S4U)	delegation_discovery.py	collectors/delegation_collector.py → attack_paths/delegation_paths.py

Capability	Original Location	New Location
RBCD abuse	delegation_discovery.py	collectors/delegation_collector.py → attack_paths/delegation_paths.py
Bloodhound graph collection	bloodhound_collection.py	collectors/bloodhound_collector.py
DA / HVT identification	bloodhound_collection.py	analyzers/privilege_analyzer.py
ACL attack paths	bloodhound_collection.py	attack_paths/acl_paths.py
Password spray suggestions	multiple phases	attack_paths/privilege_escalation_paths.py
Remediation recommendations	ad_module.py	reporting/remediation_reporter.py